

Ctrl + X to Exit

```
pi@raspberrypi ~ $ sudo minicom -b 9600
-o -D /dev/ttyAMA0
```

// This will set the baud rate of the port: ttyAMA0 to 9600

```
pi@raspberrypi ~ /etc $ cd
```

// This should change directory to main Pi folder

```
pi@raspberrypi ~ $ sudo reboot
```

// This should restart the system

Now the serial port of Raspberry Pi is disengaged from the boot sequence and can be used to communicate with XBee module. The default baud rate of the ttyAMA0 is 115200, but a baud rate of 9600 is required for XBee. You can change the baud rate in minicom software as given below:

```
pi@raspberrypi ~ $ sudo minicom -b 9600
-o -D /dev/ttyAMA0
```

// This will set the baud rate of the port ttyAMA0 to 9600

A window, as shown in Fig. 9, will come up. Press Ctrl + A keys followed by Q on the keyboard to exit without resetting.

Python code

Start your Raspberry Pi and create a new 'XBeeTest' folder. For this tutorial, the libraries used are 'serial' and 'time.' Create a new file using nano editor as given below:

```
$ pi@raspberrypi ~ $ cd /XBeeTest
pi@raspberrypi ~ /XBeeTest $ sudo nano
serial_XBee.py
```

Here serial_XBee.py is the name of the project. Type the following code:

```
import serial
// For serial communication
import time
// For delay function
ser = serial.Serial('/dev/ttyAMA0')
// Set ser as object
ser.baudrate = 9600
// Set baud rate to 9600
ser.write("A")
// Send character 'A'
time.sleep(1)
// Give 1 second delay
ser.write("B")
// Send character 'B'
```

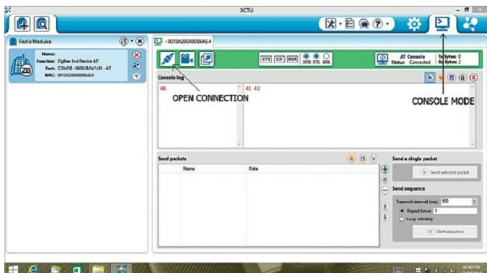


Fig. 10: Receiver console

```
time.sleep(1)
// Give 1
second delay
ser.close()
// Close
communication
```

To save this code, press Ctrl + O keys followed by Ctrl + X.

This code sends character 'A' to the console (refer Fig. 10) of the receiver end, pauses for a second and then sends character 'B.'

Construction and testing

Now it is time to test our project. After saving the code and connecting coordinator mode XBee module to your Raspberry Pi, boot the Raspberry Pi and from the terminal write the following commands to execute your program. The destination folder is XBeeTest and file name is serial_XBee.py

```
pi@raspberrypi ~ $ cd /XBeeTest
// Change Directory to XBeeTest
```

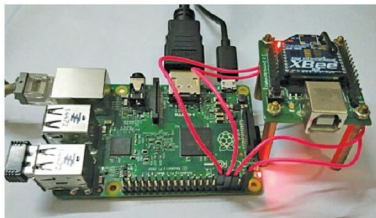


Fig. 11: Prototype of data sending unit

```
pi@raspberrypi /XBeeTest
$ sudo python serial_XBee.py
```

// Run serial_XBee.py Program

This program should send character 'A,' pause for a second and then send character 'B.' Note that, as soon as you power up both the XBee modules, ASSI (Association Indication) LED should light up. In this case, it is a yellow LED as seen in the prototype (Fig. 11).

Now to confirm reception of data, hook up your end device XBee module to your computer, open the console mode in XCTU, start serial communication with XBee module. You should get the output on your screen as shown in Fig. 10. **EFY**

EFY Note

The source code of this project is included in this month's EFY DVD and is also available for free download at source.efymag.com



Anav Bansal is pursuing B.E. in electronics and communication from UIET PUSPGRC, Hoshiarpur